DKTE College Training

CTE

- CTE is simplified -- Derived Table / Inline View

```
WITH ctename AS (
    SELECT ...
)
SELECT * FROM ctename ...;
```

- Find emps in each category (Poor < 1500, 1500 <= Mid <= 2500, 2500 < Rich)

```
WITH cte AS (
    SELECT empno, ename, sal, CASE
    WHEN sal < 1500 THEN 'Poor'
    WHEN sal BETWEEN 1500 AND 2500 THEN 'Mid'
    ELSE 'Rich'
    END cat
    FROM emp
)
SELECT cat, COUNT(empno) cnt
FROM cte
GROUP BY cat;
```

UNION vs UNION ALL

- UNION -- remove duplicate from output of two queries while combining them.
- UNION ALL -- combine output of two queries.

Recursive CTE

- Within CTE we can SELECT from that CTE.
- Example: Print numbers 1 to 4.

```
WITH RECURSIVE cte(n) AS (
    (SELECT 1)
    UNION ALL
    (SELECT (n+1) FROM cte WHERE n <= 3)
)
SELECT * FROM cte;
```

- Example: Fibonacci Series

```
WITH RECURSIVE fib(a, b) AS (
    (SELECT 0, 1)
    UNION ALL
    (SELECT b, a + b FROM fib WHERE b < 100)
)
SELECT a Term FROM fib;
```

- Find years in which emps were recruited.

```
SELECT DISTINCT(YEAR(hire)) yr FROM emp;
```

- Find years between 1975 to 1985 in which emps were not recruited.

```
WITH RECURSIVE cte_yrs(y) AS (
    (SELECT 1975)
    UNION ALL
    (SELECT y+1 FROM cte_yrs
    WHERE y < 1985)
)
SELECT * FROM cte_yrs;

WITH RECURSIVE cte_yrs(y) AS (
    (SELECT 1975)
    UNION ALL
    (SELECT y+1 FROM cte_yrs
    WHERE y < 1985)
)
SELECT * FROM cte_yrs
WHERE y NOT IN (SELECT YEAR(hire) FROM emp);

WITH RECURSIVE cte_yrs(y) AS (
    (SELECT 1975)
    UNION ALL
    (SELECT y+1 FROM cte_yrs
    WHERE y < 1985)
)
(SELECT y FROM cte_yrs)
EXCEPT
(SELECT DISTINCT(YEAR(hire)) y FROM emp);
```

Data Structures

- Data Structures in Java
  - Stack
  - Queue
  - Linked List
  - ~~Tree~~

- ~~Graph~~

Linked List in Java

- Linked List is List of Items Linked to Each Other.
- Each Item in List is called as "Node".
- Each Node in List contains some data and pointer to next Node.
-

```
class ListNode {
    int val;
    ListNode next;
    public ListNode(int v) {
        val = v;
        next = null;
    }
};
```

```
class LinkList {
    ListNode head;
    public LinkList() {
        head = null;
    }
    public void addLast(int v) {
        ListNode nn = new ListNode(v);
        if(head == null)
            head = nn;
        else {
            ListNode trav = head;
            while(trav.next != null)
                trav = trav.next;
            trav.next = nn;
        }
    }
    public void display() {
        ListNode trav = head;
        while(trav != null) {
            System.out.print(trav.val + ", ");
            trav = trav.next;
        }
        System.out.println();
    }
}
```

Assignments

1. https://leetcode.com/problems/middle-of-the-linked-list

2. https://leetcode.com/problems/delete-the-middle-node-of-a-linked-list
3. https://leetcode.com/problems/linked-list-cycle
4. https://leetcode.com/problems/reverse-linked-list/description/
5. https://leetcode.com/problems/reverse-linked-list-ii